Decision-aligned eviction-value prediction for learning-augmented caching

Soroush Vahidi[a]

[a] *Ying Wu College of Computing, New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

Caching with predictions is useful only when those predictions change the eviction that is actually made. We study unweighted online paging as candidate-level scoring: at each full-cache miss, `evict_value_v1` estimates a finite-horizon replay loss for every resident item under a fixed continuation rule and evicts the apparent least-harmful candidate. Under a matched protocol against LRU, SIEVE, FIFO-Reinsertion, LRB, 3L-Cache, CACHEUS, and HALP, the method loses on a clear majority of 21 cells against every baseline. A full-pipeline objective ablation likewise finds eviction-loss worst or tied-worst, but a matched common-model control that changes only the training objective does *not* support blaming that label: eviction-loss is nominally best by total misses. The learned scorer agrees with the exact target on 97.5% of decisions, yet every measured non-LRU exact-oracle decision has multiple optimal actions (mean optimal-set fraction ≈ 0.991). A deterministic exact oracle loses to LRU on 18 of 21 cells (+81,750 misses), but choosing the LRU-most minimizer never loses (-413 misses); the deficit is a tie-breaking confound, not proof that the exact target intrinsically loses to LRU. Continuation-label correction helps in most but not all cells; a generic DAgger-style shift correction does not. Wall-clock cost is orders of magnitude above lightweight baselines. The leading supported limitation is action-underdetermined supervision, not a deployment-ready replacement policy.

Keywords: learning-augmented caching, online paging, cache replacement, eviction-value prediction, caching baselines

Email address: `sv96@njit.edu` (Soroush Vahidi)

1. Introduction

1.1. Problem and Research Question

In online paging, a full-cache miss requires choosing which resident item to evict without future requests [1, 2]. Learning-augmented caching shows that predictions can help when they are informative and used carefully, but gains are regime-dependent and vanish when advice is noisy or weakly tied to the replacement objective [3, 4, 5, 6, 7, 8]. The operational question is therefore candidate selection: which currently cached item is safest to remove, not merely whether a hint should be trusted.

This paper tests a seemingly natural answer—supervise each candidate by the finite-horizon downstream miss cost of evicting it—and asks whether *decision alignment* is sufficient for useful online control. We distinguish three properties and evaluate them separately: alignment (the label is the eviction cost, not an indirect proxy); *decision informativeness* (the label actually separates candidates); and *deployment efficiency* (online scoring cost is acceptable). Alignment is built in by construction; the other two are empirical. Scope is unweighted paging with unit miss cost. We claim neither a competitive-ratio guarantee nor a deployment-ready replacement policy.

1.2. Main Contributions

The main contributions of this paper are as follows.

1. We formulate paging as candidate-level scoring and instantiate a finite-horizon eviction-value predictor, `evict_value_v1`.
2. Under a matched protocol (identical hashed traces, capacities, windows, and miss accounting) against seven classical and learned baselines, including LRB and 3L-Cache, `evict_value_v1` does not outperform any baseline.
3. A full-pipeline ablation against reuse-distance, next-arrival, and pairwise-preference supervision finds eviction-loss worst or tied-worst. A matched common-model control that changes only the training objective does *not* support blaming that objective: eviction-loss is nominally best by aggregate misses.
4. Mechanistic diagnostics (exact oracle, degeneracy audit, learned/exact agreement, and tie-aware replay) disfavor gross fitting failure and insufficient data. The horizon-4 target is highly action-underdetermined;

the deterministic exact-oracle deficit versus LRU is a tie-breaking confound. Continuation-label mismatch is a partial, family-dependent contributor; generic DAgger-style shift correction does not improve online misses.

5. Controlled timing shows per-decision cost orders of magnitude above lightweight baselines. Combined with the negative miss-ratio result, the present implementation is neither performance-competitive nor deployment-efficient.

1.3. Related Work

Belady's offline rule and competitive paging frame the core difficulty: eviction quality depends on future reuse, yet the choice is online [1, 2]. Learning-augmented paging typically uses next-arrival or action advice with robustness when advice is wrong [3, 9, 10, 5, 4, 6, 7, 11]. We share the robustness concern but study a candidate-level cost target rather than hint-trust or expert switching.

Learned replacement systems supervise from future-derived labels: PARROT imitates Belady; LRB and Raven predict Belady-guided reuse; Mockingjay estimates reuse times; HALP learns candidate preferences; 3L-Cache combines a lightweight scorer with bidirectional sampling; CACHEUS mixes eviction experts [12, 13, 14, 15, 16, 17, 18]. LRB, 3L-Cache, CACHEUS, and HALP are compared directly in Section 3.4. Proxies in this literature (oracle actions, reuse distance, next arrival, pairwise order) still require translation into an eviction. Our target is a counterfactual finite-horizon miss count. Structural alignment does not imply informativeness; we use decision-focused/predict-then-optimize language only as a conceptual link, including the observation that multiple optima are a form of degeneracy [19, 20, 21, 22].

MUSTACHE forecasts pages over a horizon; we supervise eviction cost instead [23]. MAT, GL-Cache, LAH/S4-FIFO, and Merlin reduce hot-path inference; S3-FIFO and SIEVE show that simple queues can be competitive [24, 25, 26, 27, 28, 29]. Our comparison includes SIEVE and FIFO-Reinsertion as lightweight anchors.

2. Main Method

2.1. System Model and Preliminaries

We study unweighted paging. Let $\sigma = (r_1, \ldots, r_T)$ with $r_t \in \mathcal{P}$. The cache has capacity k; $C_t \subseteq \mathcal{P}$ is the content before r_t. A miss with $|C_t| = k$

requires evicting one resident item [1, 2]. Each miss has unit cost, so

$$\mathrm{Cost}(\sigma) = \sum_{t=1}^{T} m_t. \tag{1}$$

The eviction candidates are $\mathcal{V}_t = C_t$. For candidate $q \in \mathcal{V}_t$ and horizon $H \geq 1$,

$$L_H(q,t) = \sum_{\tau=t+1}^{t+H} m_\tau^{(q,t)} \tag{2}$$

is the miss count over the next H requests after evicting q at t and replaying under a fixed continuation rule (here, lightweight LRU-style replay). This is a local proxy, not an offline optimum. We keep a single definition of L_H; the learned policy approximates it.

2.2. Eviction-Value Prediction Framework

Each candidate receives a feature vector $x(q,t) \in \mathbb{R}^d$ summarizing the request, cache state, and candidate context. Features fall into six groups: request-side context, candidate recency/reuse, predictor–LRU comparison, cache-state summary, predictor/recency disagreement, and recent activity. The accompanying code release contains the full feature specification. The scorer is

$$\widehat{L}_H(q,t) = f_\theta\big(x(q,t)\big), \tag{3}$$

and the victim is

$$q_t^* = \arg\min_{q \in \mathcal{V}_t} \widehat{L}_H(q,t). \tag{4}$$

An optional early-return guard that can temporarily delegate to a conservative baseline exists in the accompanying code; it is unvalidated and unused in any quantitative claim (Section 4.2).

2.3. Algorithmic Workflow

Figure 1 summarizes the pipeline. Traces are converted to a common format. At each full-cache miss, one supervised example per candidate is built from $x(q,t)$ and $L_H(q,t)$. A scorer is fit; online, all k residents are scored and the arg min of (4) is evicted. Learned policies and baselines share the same simulator.

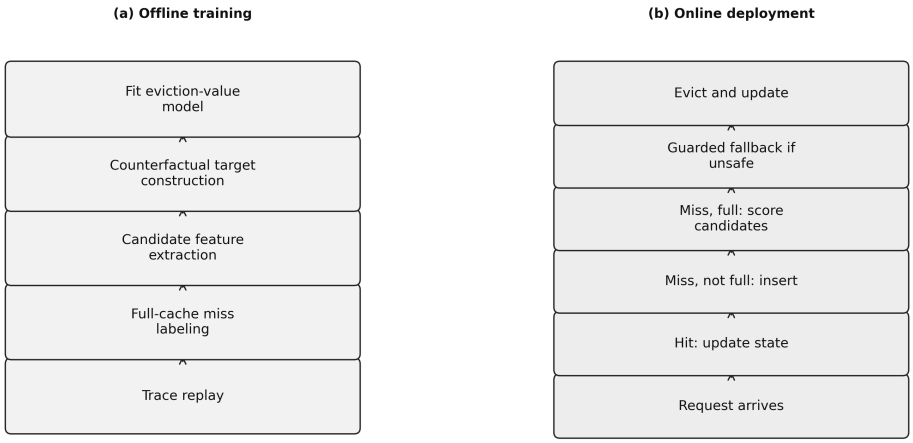

Figure 1: Eviction-value workflow: offline candidate-level labels, supervised scoring, and online arg min eviction. An optional fallback layer is shown only for conceptual completeness; it is not evaluated here.

3. Experiments and Discussion

3.1. Experimental Setup

All policies run in one simulator on the same processed traces, capacities, request order, and loader. The study has two parts: offline scorer selection on L_H, then online replay of the induced policy. Offline target quality need not imply online miss-ratio gains. The setting is unweighted paging; the primary metric is total misses. Claims are restricted to the reported artifacts.

3.2. Datasets, Baselines, and Evaluation Metrics

The primary comparison uses LRU, SIEVE, and FIFO-Reinsertion as classical anchors [1, 2, 29], and LRB, 3L-Cache, CACHEUS, and HALP as learned systems [13, 17, 18, 16] (Section 3.4). PredMk, T&D, BO/LRU, and REST appear only as historical context below [3, 4, 6]. Table 1 lists workloads; BrightKite and Citi Bike are transformed event streams, not native cache traces. Table 2 lists policies. Downstream misses are decisive; offline regret and Top-1 are diagnostic.

3.3. Offline Ablation and Model Selection

Table 3 selects the scorer. Tree models beat ridge on regret and Top-1 at every horizon. $H = 4$ is strongest; $H = 8$ and $H = 16$ degrade,

Table 1: Trace families in the reported evaluation.

Family	Source	Transformation	Role
BrightKite	SNAP check-ins [30]	Venue ID as item; original order.	Locality-stress stream.
Citi Bike	Bike-share trips [31]	Start-station ID as item.	Locality-stress stream.
CloudPhysics	Block I/O [32]	Read-only LBN→4 KiB page.	Storage-derived.
MetaCDN	CacheLib/cacheMon CDN [32]	Manifest object IDs.	Cache-derived.
MetaKV	CacheLib/cacheMon KV [32]	Manifest keys.	Cache-derived.
Twemcache	Twitter cache [33, 34]	Anonymized keys.	Production cache.
Wikimedia	Pageviews [35]	Pageview IDs (proxy, not a CDN trace).	Event-stream proxy.

including validation MAE/RMSE and regret-vs-oracle, in every validation family (BrightKite, CloudPhysics, MetaCDN, Twemcache, Wikimedia). A short horizon ties the label to the scored eviction; longer horizons mix in later continuation decisions. This is the empirical $H = 4$ justification, not a claim of online superiority.

Table 4 reports the leave-one-family-out matched-protocol miss gap versus LRU by family on the primary scored window. Wikimedia is degenerate. CloudPhysics and MetaKV stay near LRU (MetaKV at capacity 128 is a 0.03% improvement). The largest gaps are MetaCDN at capacity 32 and Twemcache at capacity 128; BrightKite and Citi Bike remain worse than LRU at every capacity. A superseded single-split sweep that also included PredMk, T&D, BO/LRU, and REST is not primary evidence.

The aggregate matched comparison against all seven baselines is next.

3.4. Matched Comparison Against Learned Cache-Replacement Baselines

This section is the primary evidence: leave-one-family-out training of `evict_value_v1` and a matched comparison to LRB, 3L-Cache, CACHEUS, HALP, LRU, SIEVE, and FIFO-Reinsertion. Table 4 is the family-level view of the same protocol versus LRU.

Table 2: Policies in the empirical comparison. Learned systems are evaluated only in the matched protocol of Section 3.4.

Policy	Role in evaluation
Classical baselines	
LRU	Recency reference [1].
SIEVE	CLOCK-family FIFO with an advancing hand [29].
FIFO-Reinsertion	Second-Chance reinsertion at queue head [29].
Predictive / robust references (historical context only)	
PredMk	Prediction-aware marker paging [3].
BO/LRU	Blind-oracle/LRU combiner [6].
T&D	Trust-and-doubt control [4].
REST	Regret-driven switch between a predictor rule and LRU.
Learned baselines (matched protocol)	
LRB	Belady-guided CDN eviction; independent reimplementation [13].
3L-Cache	Bidirectional sampling scorer; independent reimplementation [17].
CACHEUS	Expert mixture; official-source wrapper [18].
HALP	Candidate preference learning; independent reimplementation, no official public code [16].
Proposed method	
EV (`evict_value_v1`)	Candidate-level finite-horizon eviction-value predictor.

Table 3: Offline ablation: lower regret and Top-1 error are better.

Horizon	Model	Val. regret	Test regret	Val. Top-1	Test Top-1
4	hist_gb	0.0091	0.0055	0.0174	0.0316
4	random_forest	0.0134	0.0046	0.0154	0.0264
4	ridge	0.0649	0.0150	0.0503	0.0832
8	hist_gb	0.0203	0.0104	0.0157	0.0362
8	random_forest	0.0240	0.0120	0.0177	0.0267
8	ridge	0.0880	0.0384	0.0377	0.0795
16	hist_gb	0.0394	0.0196	0.0203	0.0365
16	random_forest	0.0431	0.0178	0.0157	0.0249
16	ridge	0.1166	0.0850	0.0277	0.0826

Table 4: Matched-protocol `evict_value_v1` miss gap vs. LRU by family (leave-one-family-out; 40,000 scored requests). Positive is worse.

Trace family	Cap. 32	Cap. 64	Cap. 128
BrightKite	+8.69%	+6.37%	+12.82%
Citi Bike	+5.55%	+2.54%	+9.45%
CloudPhysics	+0.74%	+1.35%	+2.84%
MetaCDN	+25.85%	+4.08%	+5.73%
MetaKV	+2.31%	+0.00%	−0.03%
Twemcache	+3.27%	+4.51%	+15.68%
Wikimedia[†]	+0.00%	+0.00%	+0.00%

[†]All policies miss every request (working set exceeds capacity). Not informative for ranking.

3.4.1. Corrected training protocol

`evict_value_v1` is retrained under leave-one-family-out: for each of seven families, the model is trained on the other six (five train, one validation-only), and the held-out family contributes no rows to training, validation, hyperparameter, feature, horizon, seed, or early-stopping choices.

3.4.2. Matched evaluation protocol

All eight policies use identical traces (SHA-256), capacities $\{32, 64, 128\}$, history prefix $[0, 10{,}000)$, scored suffix $[10{,}000, 50{,}000)$, unit-object slots, and hit/miss accounting. Matching is evaluation-side only: LRB, 3L-Cache, and CACHEUS adapt online on the scored stream; HALP trains on each trace's own prefix; LRU/SIEVE/FIFO-Reinsertion are parameter-free; `evict_value_v1` is leave-one-family-out. No policy sees future requests or another family's data.

3.4.3. Results

Table 5 reports, for each baseline, the mean miss ratio over the 21 matched (family, capacity) cells, the number of cells `evict_value_v1` wins, loses, and ties against that baseline, and the relative mean difference in miss ratio. `evict_value_v1`'s own mean primary miss ratio across all 21 cells is 0.700463.

Table 5: Matched comparison, 21 cells. Positive relative difference: `evict_value_v1` worse.

Baseline	Baseline mean	EV wins	Baseline wins	Ties	Rel. diff.
LRB	0.6877	5	13	3	+1.85%
3L-Cache	0.6926	5	13	3	+1.13%
CACHEUS	0.6756	3	15	3	+3.68%
HALP	0.6762	1	17	3	+3.59%
LRU	0.6728	1	16	4	+4.12%
SIEVE	0.6875	4	14	3	+1.89%
FIFO-Reinsertion	0.6733	2	16	3	+4.04%

`evict_value_v1` loses on a majority of cells against every baseline (13–17 losses; relative miss-ratio gap +1.13% vs. 3L-Cache to +4.12% vs. LRU). It offers no matched advantage over LRB or 3L-Cache.

3.4.4. Implementation-fidelity caveats

LRB and 3L-Cache are independent reimplementations (3L-Cache uses an untuned default batch size). CACHEUS wraps the authors' engine; the upstream clone is not live-verifiable here. HALP has no official public code and is a lower-fidelity supporting comparison. Caveats concern algorithm fidelity, not the matched inputs in Table 5.

3.5. Direct Comparison Against Alternative Supervision Objectives

Section 2.2 treats eviction-loss as structurally decision-aligned: the label is a candidate-level miss count, not an indirect proxy. Whether that alignment is empirically preferable to reuse distance, next arrival, or pairwise preference is tested rather than assumed.

Under the same leave-one-family-out protocol, we swap only the candidate-level label. Table 6 is the full pipeline.

Table 6: Full-pipeline supervision-objective comparison, 21 cells (lower misses better).

Objective	Total misses	Mean miss ratio
Eviction loss (proposed)	601,569	0.7162
Next arrival	573,059	0.6822
Reuse distance	571,456	0.6803
Pairwise preference	565,127	0.6728

Eviction-loss is worst of the four in the deployed pipeline, and worst or tied in every family (Wikimedia ties at full misses). That ranking does not isolate the training objective. A matched common-model control holds architecture, features, folds, windows, and seed fixed and changes only the label; pairwise uses a corrected preference orientation. Table 7 reports the result.

Table 7: Matched common-model V2: only the training objective changes. 21 cells.

Objective	Total misses	Mean miss ratio
Eviction loss	571,976	0.6809
Pairwise preference	577,339	0.6873
Reuse distance	615,850	0.7332
Next arrival	627,392	0.7469

Eviction-loss is nominally best by totals and macro mean miss ratio; pairwise is $\approx 0.9\%$ behind on totals but strongest by per-cell rank. Eviction-loss is not materially worse. The matched control does *not* support blaming the eviction-loss training objective. Table 6 remains a valid pipeline ranking, not an isolated objective-causality result. Section 3.6 examines action-underdetermination.

3.6. Mechanistic Diagnosis of the Eviction-Loss Target

The matched comparison and pipeline ablation are negative; Table 7 shows the training objective is not the isolated cause. We diagnose the target on the same 21 cells.

Is inaccurate prediction the cause? An exact horizon-4 solver for the same target agrees with the learned scorer on 97.53% of decisions (macro mean *set-aware* agreement): the learned victim usually lies in the exact optimal set. Mean target regret is 0.0247 and the positive-regret fraction is 0.0247; learned misses (601,569) are within 6.4% of LRU (565,126). Because that set is itself extremely large (below), set-aware agreement is weak evidence of candidate discrimination. It does disfavor *gross* fitting failure—the scorer rarely leaves the set the target defines.

Is the target useful if optimized perfectly? Under deterministic tie-breaking (minimum candidate identifier), the exact oracle has mean miss ratio 0.770090 vs. 0.672769 for LRU: 0 wins / 3 ties / 18 losses, +81,750 misses (646,876 vs. 565,126). That instantiation is a poor online policy; it is not a statement about every way of resolving the same optimal sets.

A tie-aware control varies only how exact minimizers are chosen. Every measured non-LRU oracle decision has multiple optima (tied-decision fraction 1.0 on all 168 non-LRU rows; mean fraction of all-candidates-tied decisions ≈ 0.649; mean optimal-set fraction ≈ 0.991; Wikimedia fully degenerate). Choosing the LRU-most minimizer never loses to LRU (16/5/0; -413 misses; 564,713 vs. 565,126). Choosing the MRU-most minimizer matches the deterministic deficit (0/3/18; +89,135). Uniform random among minimizers (five seeds) usually loses to LRU (7/15/83 over 105 rows) but usually beats the deterministic oracle (90/15/0). The deterministic deficit is therefore a tie-breaking confound. It does *not* show that the exact H4 target intrinsically loses to LRU, nor that $H = 4$ supervision improves LRU as a learned policy. It shows that a recency secondary rule among exact minimizers can remove the identifier-tie deficit.

Why so underdetermined? Two diagnostics agree qualitatively and should not be treated as interchangeable point estimates. In the tie-aware replay above, the mean fraction of decisions in which every candidate is tied is ≈ 0.649 and the mean optimal-set fraction is ≈ 0.991. An independent strict-preference audit of the same $H = 4$ target reports unique-winner fraction 0.0, multiple-optimum fraction 1.0, 76.4% all-tied decisions, and mean optimal-set fraction 0.9949. The numerical gap reflects different instrumentation (online oracle replay vs. a strict-preference diagnostic), not a contradiction. Across 41,615,776 candidate-residency observations at 565,126 decisions, $P(T > 4) = 0.9939$ overall and 0.9793 even conditioned on eventual reuse. Most consequences lie outside $H = 4$, so the target often cannot distinguish candidates.

Summary. A learning-curve check at 1–50% of candidate examples shows no material target-quality gain, which disfavors a sample-size explanation. Gross fitting failure is likewise disfavored, with the caveat above that set-aware agreement does not imply discrimination inside a near-full optimal set. The eviction-loss *training objective* is not supported as the culprit (Table 7). The strongest supported diagnosis is action-underdetermined $H = 4$ supervision. Tie degeneracy is not claimed to explain every aspect of learned underperformance. Section 3.7 tests a secondary continuation/deployment contributor.

3.7. Continuation-Mismatch and Distribution-Shift Ablations

Two controlled tests address continuation labels vs. generic state shift, previously speculative.

C0 is LRU. *C1* trains π_1 on LRU-continuation labels (Eq. (2)) and deploys π_1 recursively. *C2* retrains π_2 on frozen-π_1 continuation and deploys π_2; that is the only C1→C2 change. C2 improves on 13 cells, ties 3 (Wikimedia), worsens 5 (macro mean -0.0102; 601,569 → 592,970 misses vs. C0 565,126). BrightKite at capacity 32 regresses $+0.2433$. Continuation mismatch is *partially supported and regime-dependent*, not a full explanation.

A one-step DAgger-style mix of on- and off-policy states reduces a generic shift index in 16 of 21 cells, yet misses improve in 2, tie in 3, and worsen in 16 (macro $+0.0094$; 591,604 → 599,537). In 13 of 18 informative cells the shift metric improves while misses worsen. The tested generic correction is a negative result; it does not imply that distribution shift is absent.

3.8. Discussion and Synthesis

The canonical interpretation is as follows. Alignment of L_H with the eviction action does not imply informativeness or efficiency. Offline, short-horizon nonlinear scorers learn the target (Section 3.3). Online, `evict_value_v1` loses to all seven matched baselines (Section 3.4). In the deployed pipeline eviction-loss is worst of four objectives (Table 6), but matched Common-Model V2 does not support blaming that training objective (Table 7). Target/tie degeneracy is strongly supported: every measured non-LRU exact decision has multiple optima, and deterministic exact-oracle failure vs. LRU is tie-confounded (Section 3.6). Exact-target intrinsic inferiority is not supported. Continuation mismatch is a partial, family-dependent contributor; the tested DAgger correction does not help (Section 3.7). The current implementation is neither miss-competitive nor deployment-efficient.

3.9. Overhead and Scalability

Offline label construction required about 10.3 hours and 96 GB (662 shards); fitting all horizon/model configurations took about 7–8 minutes. Online, each miss scores all k residents ($O(k)$ inferences) versus $O(1)$ for LRU, SIEVE, and FIFO-Reinsertion. A controlled campaign on a dedicated node—7 families, 3 capacities, 4 policies, 5 repetitions (420 runs)—is in Table 8.

Table 8: Controlled timing: per-request wall-clock (μs), $n = 105$ per policy.

Policy	Mean (μs)	Median (μs)	95% CI	Slowdown
LRU	4.68	4.59	[4.45, 4.91]	1.00×
FIFO-Reinsertion	5.17	5.14	[4.96, 5.38]	1.10×
SIEVE	9.52	8.85	[8.91, 10.14]	2.03×
HALP (causal)	870.66	920.14	[773.47, 967.86]	186.02×

Lightweight policies remain single-digit microseconds; causal HALP is 186× LRU. `evict_value_v1` was not in this 5-repetition campaign. A single held-out measurement is 35,290 μs/request on BrightKite at capacity 32 (about $10^4\times$ LRU). It has no CI and is not statistically comparable to Table 8. Qualitatively, the implementation is not wall-clock competitive.

3.10. Practical Significance

`evict_value_v1` is not a replacement for LRU, SIEVE, FIFO-Reinsertion, or the learned baselines in latency-sensitive deployments; no validated de-

ployment scenario is claimed. The contribution is diagnostic: a controlled test of a finite-horizon candidate-level target, showing that informativeness and cost—not merely alignment—must hold. Future practical redesign would need a more informative target and cheaper inference [24, 25, 26, 17, 27, 28, 29].

4. Conclusions and Future Research

4.1. Summary of Findings

Candidate-level $H = 4$ eviction-value scoring is learnable offline and unsuccessful online. See Section 3.8 for the mechanistic synthesis; we do not repeat it here. Decision alignment without informativeness and efficiency is insufficient. The negative result is for this target, horizon, and pipeline, not for all candidate-level learning.

4.2. Limitations

- Unweighted unit-size paging only.
- Finite $H = 4$ local proxy, not an offline-optimal rule.
- Implementation fidelity: 3L-Cache untuned batch size; CACHEUS provenance; HALP low-to-medium reimplementation (Section 3.4).
- 21 cells, fixed seeds, descriptive paired results; BrightKite/Citi Bike are event streams; Wikimedia is a pageview proxy; 50k-request windows.
- Family-level LRU gaps in Table 4 use the same matched protocol as Section 3.4. Capacity 256 is unevaluated.
- `evict_value_v1` timing is a single run, not in the 5-repetition campaign.
- No validated fallback; an optional early-return guard exists in the accompanying code and is unused for claims.
- Single-author revision; audits do not replace independent replication.
- Diagnostics constrain interpretation; they do not isolate every downstream cause of underperformance.

4.3. Future Research Directions

Priorities are informative, tie-aware targets; adaptive/longer horizons or terminal correction; iterative continuation correction; selective inference; weighted/variable-size caching; and a validated fallback. Ranking supervision remains open, but not because eviction-loss is a uniquely harmful matched objective. Capacity-256 sweeps remain undone.

Acknowledgements

The author thanks Professor Ioannis Koutis for his guidance and support throughout this work. The author also acknowledges the use of the Wulver high-performance computing system at the New Jersey Institute of Technology for part of the experimental evaluation.

Funding

The author received no specific funding for this work.

AI Declaration

During the preparation of this manuscript, the author used ChatGPT for writing assistance and manuscript refinement, and Perplexity AI for literature exploration and background research. During the implementation stage of the project, the author used Cursor, ChatGPT Codex, and GitHub Copilot for coding assistance, and Gemini for limited recommendations, including assistance with auditing repository artifacts and checking internal consistency between the manuscript and the underlying code and results. All AI-assisted text, code, and analysis were independently verified by the author against the repository's artifacts before inclusion, including dedicated schema-validation, replay-artifact, and negative-result audits documented in the repository; AI assistance was not used to introduce any positive result that is not directly supported by these verified artifacts. The author takes full responsibility for the content of the manuscript, the code, and the reported results.

Data Availability

The datasets used in this study are cited in the manuscript and were obtained from their respective public or documented sources. The repository

associated with this work contains the code used for data preparation, experiment execution, and analysis: `https://github.com/SoroushVahidi/Augmented-caching`. Access to specific datasets remains subject to the terms, licenses, and availability conditions of the original data providers.

Code Availability

The code used to prepare the data, train the models, run the baseline comparisons, and generate the experimental artifacts is available at: `https://github.com/SoroushVahidi/Augmented-caching`

Declaration of Competing Interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

References

[1] L. A. Belady, A study of replacement algorithms for virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101. doi:10.1147/SJ.52.0078.

[2] A. Fiat, R. M. Karp, M. Luby, L. A. McGeoch, D. D. Sleator, N. E. Young, Competitive paging algorithms, Journal of Algorithms 12 (4) (1991) 685–699. doi:10.1016/0196-6774(91)90041-V.

[3] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, Journal of the ACM 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.

[4] A. Antoniadis, C. Coester, M. Eliáš, A. Polak, B. Simon, Online metric algorithms with untrusted predictions, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 345–355.

[5] S. Im, R. Kumar, A. Petety, M. Purohit, Parsimonious learning-augmented caching, in: Proceedings of the 39th International Conference on Machine Learning (ICML), Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 9588–9601.

[6] A. Wei, Better and simpler learning-augmented online caching, in: Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020), Vol. 176 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:17. doi:10.4230/LIPIcs.APPROX/RANDOM.2020.60.

[7] J. Chłędowski, A. Polak, B. Szabucki, K. Żołna, Robust learning-augmented caching: An experimental study, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 1920–1930.

[8] M. Mitzenmacher, S. Vassilvitskii, Algorithms with predictions, Communications of the ACM 65 (7) (2022) 33–35. doi:10.1145/3528087.

[9] D. Rohatgi, Near-optimal bounds for online caching with machine learned advice, in: Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2020, pp. 1834–1845. doi:10.1137/1.9781611975994.112.

[10] N. Bansal, C. Coester, R. Kumar, M. Purohit, E. Vee, Learning-augmented weighted paging, in: Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2022, pp. 67–89.

[11] A. Blum, C. Burch, On-line learning and the metrical task system problem, Machine Learning 39 (1) (2000) 35–58. doi:10.1023/A:1007621832648.

[12] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, J. Ahn, An imitation learning approach for cache replacement, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6237–6247.
URL https://proceedings.mlr.press/v119/liu20f.html

[13] Z. Song, D. S. Berger, K. Li, W. Lloyd, Learning relaxed belady for content distribution network caching, in: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), USENIX Association, 2020, pp. 529–544.

URL https://www.usenix.org/conference/nsdi20/presentation/song

[14] X. Hu, E. Ramadan, W. Ye, F. Tian, Z.-L. Zhang, Raven: Belady-guided, predictive (deep) learning for in-memory and content caching, in: Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '22), ACM, 2022, pp. 72–90. doi:10.1145/3555050.3569134.
URL https://doi.org/10.1145/3555050.3569134

[15] I. Shah, A. Jain, C. Lin, Effective mimicry of belady's MIN policy, in: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), IEEE, 2022, pp. 558–572. doi:10.1109/HPCA53966.2022.00048.

[16] Z. Song, K. Chen, N. Sarda, D. Altinbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, R. Gummadi, Halp: Heuristic aided learned preference eviction policy for youtube content delivery network, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, 2023, pp. 1149–1163.
URL https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu

[17] W. Zhou, Z. Niu, Y. Xiong, J. Fang, Q. Wang, 3l-cache: Low overhead and precise learning-based eviction policy for caches, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, 2025, pp. 237–254.
URL https://www.usenix.org/conference/fast25/presentation/zhou-wenbin

[18] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, G. Narasimhan, Learning cache replacement with cacheus, in: 19th USENIX Conference on File and Storage Technologies (FAST 21), USENIX Association, 2021, pp. 341–354.
URL https://www.usenix.org/conference/fast21/presentation/rodriguez

[19] A. N. Elmachtoub, P. Grigas, Smart "predict, then optimize", Management Science 68 (1) (2022) 9–26. doi:10.1287/mnsc.2020.3922.

[20] B. Wilder, B. Dilkina, M. Tambe, Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 1658–1665. doi:10.1609/aaai.v33i01.33011658.

[21] S. Shah, K. Wang, B. Wilder, A. Perrault, M. Tambe, Decision-focused learning without decision-making: Learning locally optimized decision losses, in: Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022.
URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/0904c7edde20d7134a77fc7f9cd86ea2-Abstract-Conference.html

[22] O. E. Balghiti, A. N. Elmachtoub, P. Grigas, A. Tewari, Generalization bounds in the predict-then-optimize framework, Mathematics of Operations Research 48 (4) (2023) 2043–2065. doi:10.1287/moor.2022.1330.
URL https://doi.org/10.1287/moor.2022.1330

[23] G. Tolomei, L. Takanen, F. Pinelli, Mustache: Multi-step-ahead predictions for cache eviction, CoRR abs/2211.02177 (2022).
URL https://arxiv.org/abs/2211.02177

[24] D. Yang, D. S. Berger, K. Li, W. Lloyd, A learned cache eviction framework with minimal overhead, arXiv preprint arXiv:2301.11886 (2023). doi:10.48550/arXiv.2301.11886.
URL https://arxiv.org/abs/2301.11886

[25] J. Yang, Z. Mao, Y. Yue, K. V. Rashmi, GL-Cache: Group-level learning for efficient and high-performance caching, in: 21st USENIX Conference on File and Storage Technologies (FAST '23), USENIX Association, 2023, pp. 115–134.
URL https://www.usenix.org/conference/fast23/presentation/yang-juncheng

[26] H. Xia, W. Nixon, B. D. Marthen, P. Bhandari, J. Yang, Learning-augmented heuristics: Simple yet smart, robust and interpretable cache eviction, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26), USENIX Association, 2026, pp. 2241–2259.

URL https://www.usenix.org/conference/osdi26/presentation/xia

[27] L. Li, J. Guo, Y. Fan, J. Wu, Z. Wang, J. Zhang, Y. Tamir, X. Wang, Y. Luo, D. Zhou, Merlin: An efficient adaptive cache eviction algorithm via fine-grained characterization, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26), USENIX Association, 2026, pp. 2223–2239.
URL https://www.usenix.org/conference/osdi26/presentation/li-liujia

[28] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, K. V. Rashmi, Fifo queues are all you need for cache eviction, in: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23), ACM, 2023. doi:10.1145/3600006.3613147.
URL https://doi.org/10.1145/3600006.3613147

[29] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, K. V. Rashmi, SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, 2024, pp. 1229–1246.
URL https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo

[30] E. Cho, S. A. Myers, J. Leskovec, Friendship and mobility: User movement in location-based social networks, in: Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2011, pp. 1082–1090. doi:10.1145/2020408.2020579.

[31] Citi Bike, Citi bike system data, accessed: 2026-04-10 (2025).
URL https://citibikenyc.com/system-data

[32] J. Yang, Open-source cache dataset, gitHub repository. Accessed: 2026-04-10 (2024).
URL https://github.com/cacheMon/cache_dataset

[33] Twitter, Anonymized cache request traces from twitter production, official trace repository; accessed 2026-08-13 (2020).
URL https://github.com/twitter/cache-trace

[34] J. Yang, Y. Yue, K. V. Rashmi, A large scale analysis of hundreds of in-memory cache clusters at twitter, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 191–208.
URL https://www.usenix.org/conference/osdi20/presentation/yang

[35] Wikimedia Foundation, Analytics datasets: Pageviews, accessed: 2026-04-10.
URL https://dumps.wikimedia.org/other/pageviews/readme.html